

Conventions used

There are a number of text conventions used throughout this book.

CodeInText: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: “Mount the downloaded `WebStorm-10*.dmg` disk image file as another disk in your system.”

A block of code is set as follows:

```
SELECT rolname, rolcanlogin,  
       rolconlimit, rolpassword  
FROM pg_roles  
WHERE rolname = 'luca';
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
SELECT line_number, type,  
       database, user_name,  
       address, auth_method  
FROM pg_hba_file_rules;
```

Any command-line input or output is written as follows:

```
$ sudo cat $PGDATA/rejected_users.txt
```

Bold: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: “Select **System info** from the **Administration** panel.”



Warnings or important notes appear like this.



Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: Email feedback@packtpub.com and mention the book's title in the subject of your message. If you have questions about any aspect of this book, please email us at questions@packtpub.com.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packtpub.com/submit-errata>, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packtpub.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packtpub.com>.

Share your thoughts

Once you've read *Learn PostgreSQL*, we'd love to hear your thoughts! Please [click here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?
Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/9781837635641>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

1

Introduction to PostgreSQL

PostgreSQL is a well-known open-source relational database, and its motto states what the project intends to be *the most advanced open-source database in the world*.

The main qualities that attract masses of new users every year and keep current users enthusiastic about PostgreSQL are its rock-solid stability, scalability, and safeness, as well as all the features that an enterprise-level database management system must provide.

While PostgreSQL is a relational database, its ecosystem has grown over time, providing a rich platform with extensions, tools, and languages tied together by communities spread around the world.

PostgreSQL is an open-source project and is fully developed in the open-source world. That means that there is no single entity in charge of the project and the result is that PostgreSQL is not a commercial product. In other words, PostgreSQL belongs to everyone, and anyone can contribute to it. Thanks to a very permissive BSD-style license, PostgreSQL can be used in any project or scenario, either open or closed source.

Of course, contributing to a project of that size and complexity requires experience in software development, database concepts, and, of course, a positive attitude to open-source and collaborative efforts. Being open-source in nature means that PostgreSQL will continue to live pretty much forever without the risk of a single company going out of business and sinking with the database.

The official PostgreSQL developers are generally known as the **PostgreSQL Global Development Group (PGDG)**, and they are the developers that, after discussion and coordination, implement the main features and produce new releases. The PGDG delivers a new production release once per year, usually in the last quarter of the year.

At the time of writing, PostgreSQL 16 is the latest production release of this great database engine, and as usual, efforts for the next release (PostgreSQL 17) are ongoing.

This book will focus on how you can get the best out of PostgreSQL, starting from the basics (managing users, data tables, indexes, and so on) and moving toward the most exciting and complex features (such as replicating your data to prevent disasters). We'll take a practical approach, with several examples, in order to let readers better understand every concept and acquire knowledge in a more fun and quick way. At the end, you will be able to fully administer a PostgreSQL cluster and, thanks to the resources pointed out in every chapter, you will be able to research even more features.



This book covers PostgreSQL 16, but the concepts explained in this book can also be applied to later versions (as well as to previous ones where the same features are present). In fact, while some tools could change in future releases (e.g., adding or removing some options), the basic concepts expressed in the book will remain pretty much the same without any regard to the PostgreSQL version.

This chapter will introduce you to this great open-source database starting from the project history and goals; you will learn basic PostgreSQL terminology, which is very important to help you search the documentation and understand the main error messages, in case you need to. Finally, you will see how to install PostgreSQL in different ways so that you will get a basic knowledge of how to install it on different platforms and in different contexts.

The following topics are covered in this chapter:

- PostgreSQL at a glance
- Exploring PostgreSQL terminology
- Installing PostgreSQL 16 or higher

Technical requirements

You can find the code for this chapter at the following GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>.

PostgreSQL at a glance

As a relational database, PostgreSQL provides a lot of features, and it is quite difficult to “scare” a PostgreSQL instance.

In fact, a single instance can contain more than 4 billion individual databases, each with unlimited total size and capacity for more than 1 billion tables, each containing 32 TB of data. Moreover, if there's any concern that those upper limits won't suffice, please consider that a single table can have 1,600 columns, each 1 GB in size, with an unlimited number of multi-column indexes (up to 32 columns). In short, PostgreSQL can store much more data than you can possibly think of!

While PostgreSQL can handle such huge amounts of data, that does not mean that you should use it as a dumping ground or catch-all storage: in order to perform well with certain big databases, you need to understand PostgreSQL and its features, being therefore able to organize and manage your datasets.

PostgreSQL is fully ACID-compliant (see the box below) and has a very strong foundation in data integrity and concurrency. It ships with a procedural language, named PL/PgSQL, which can be used to write reusable pieces of code, such as functions and procedures, and it supports before and after triggers, views, materialized views, partitioned tables, foreign data wrappers, multiple schemas, generated columns, and so on. All of these concepts will be explained in the forthcoming chapters.



ACID is an acronym of properties, used to indicate that the database engine provides **atomicity**, **consistency**, **isolation**, and **durability**. Atomicity means that a complex database operation is processed as a single instruction even when it is made up of different operations. Consistency means that the data within the database is always kept consistent and that it is not corrupted due to partially performed operations. Isolation allows the database to handle concurrency in the “right way”—that is, without having corrupted data from interleaved changes. Lastly, durability means that the database engine is supposed to protect the data it contains, even in the case of either software or hardware failures, as much as it can.

PostgreSQL can be extended with other embedded languages, such as Perl, Python, Java, and even Bash! And if you think the database does not provide you with enough features, you can plug in extensions to obtain different behaviors and enhancements—for instance, **Geospatial Information System (GIS)**, scheduled jobs, esoteric data types, and utilities in general. Such utilities and enhancement will not be covered in this book, but thanks to the knowledge this book provides, it will be possible to exploit the online documentation of such utilities to get the best out of them.

PostgreSQL runs on pretty much every operating system out there, including Linux, Unix, macOS X, and Microsoft Windows, and can even run on commodity hardware such as Raspberry Pi boards. There are also several cloud computing providers that list PostgreSQL in their software catalog.

Thanks to its extensive tuning mechanism, it can be adapted very well to the hosting platform. The community is responsible for keeping the database and documentation at a very high-quality level, and also, the mailing lists and IRC channels are very responsive and a valuable source for problem solutions and ideas.

In the experience of the authors, there has never been a case where PostgreSQL has not been able to adapt to an application scenario.



The PostgreSQL project has a very rich and extensive set of mailing lists that range from general topics to very specific details. It is a good habit to search for problems and solutions on the mailing list archives; see the web page at <https://www.postgresql.org/list/> to get a better idea.

A brief history of PostgreSQL

PostgreSQL takes its name from its ancestor: Ingres.

Ingres was a relational database developed by Professor Michael Stonebraker. In 1986, Professor Stonebraker started a post-Ingres project to develop new, cool features in the database landscape and named this project **POSTGRES (POST-Ingres)**. The project aimed to develop an object-relational database, where “object” means the user would have the capability to extend the database with their own objects, such as data types, functions, and so on.

In 1994, POSTGRES was released with version 4.2 and an MIT license, which opened up collaboration with other developers around the world. At that time, POSTGRES was using an internal query language named **QUEL**. Two Berkeley students, Andrew Yu and Jolly Chen, replaced the QUEL query language with the hot and cool SQL language, and the feature was so innovative that the project changed its name to Postgre95 to emphasize the difference compared to other, preceding versions.

Eventually, in 1996, the project gained a public server to host the code, and five developers, including Marc G. Fournier, Tom Lane, and Bruce Momjian, started the development of the newly branded project named **PostgreSQL**. Since then, the project has been kept in good shape and up to date.

This also means that PostgreSQL has been developed for nearly 30 years, again emphasizing the solidity and openness of the project itself. If you are curious, it is also possible to dig into the source code down to the initial commit in the open-source world:

```
$ git log 'git rev-list --max-parents=0 HEAD'
commit d31084e9d1118b25fd16580d9d8c2924b5740dff
Author: Marc G. Fournier <scrappy@hub.org>
Date:   Tue Jul 9 06:22:35 1996 +0000

Postgres95 1.01 Distribution - Virgin Sources
```

What's new in PostgreSQL 16?

PostgreSQL 16 was released on 14th September 2023. It includes a rich set of improvements, including the following:

- Several performance optimizations, ranging from internal memory allocation and management to a more parallelized-by-default behavior.
- A revised set of permissions for users and groups, including new system groups to provide specific capabilities.
- An improved configuration mechanism, to ease the inclusion of files and match users and hosts by means of regular expression.
- A more complete set of JSON functions.
- An improved logical replication engine that allows decoding even on the stand-by servers.
- A set of utility columns gained new options to fine-tune what the administrator needs to do.

As with other releases, PostgreSQL 16 also contains a set of changes aimed at making the **Database Administrator (DBA)**'s life easier—for instance, removing conflicting options and obsolete SQL terms and types. This emphasizes the fact that PostgreSQL developers do always take care of the database and its adherence to the current SQL standards.

PostgreSQL release policy, version numbers, and life cycle

PostgreSQL developers release a new major release once per year, usually around October. A *major release* is a stable version that introduces new features and possible incompatibilities with previous versions. During its life cycle, a major release is constantly improved by means of *minor releases*, which are usually bug-fixing and maintenance releases.

The PostgreSQL version number identifies the major and minor release. The version number is specified as `major.minor`; so, for instance, `16.0` indicates the first major release, 16, while `16.1` indicates the minor release, 1, of major release 16. In short, the greater the number, the more recent the version you are managing.

PostgreSQL's different major versions are incompatible, while different minor versions are compatible. What does such incompatibility mean? PostgreSQL stores data in binary format, and this format could possibly change between major versions. This means that, while you are able to upgrade PostgreSQL between minor versions on the fly, you probably will have to dump and restore your database content between major version upgrades.

The recommendation, as for much other software, is to run the most recent version of PostgreSQL available to you: PostgreSQL developers put in a lot of effort in order to provide bug-free products, but new features could introduce new bugs, and regardless of the very extensive testing platform PostgreSQL has, it is software after all, and software has bugs. Despite internal bugs, new releases also include fixes for security exploits and performance improvements, so it is a very good habit to keep up to date with your running PostgreSQL server.

Last but not least, not all PostgreSQL versions will live forever. PostgreSQL provides support and upgrades for five years after a new release is issued; after this length of time, a major release will reach its **End Of Life (EOL)** and PostgreSQL developers will no longer maintain it. This does not mean you cannot run an ancient version of PostgreSQL; it simply means this version will not get any upgrades from the official project and, therefore, will be out of date. As an example, since PostgreSQL 16 was released in 2023, it will reach its EOL in 2028. Keep in mind that running an EOL release is not only a matter of not getting new upgrades, security patches, and bug fixes; you will be on your own and you will not find help when you run into trouble.

With that in mind, we'll now introduce the main PostgreSQL terminology, as well as further useful-to-understand concepts.

Exploring PostgreSQL terminology

In order for you to understand how PostgreSQL works and follow the examples in the chapters of this book, we need to introduce the terminology used within PostgreSQL and its community of users.

PostgreSQL is a service, which means it runs as a daemon on the operating system; a running PostgreSQL daemon is called an instance. A PostgreSQL instance is often called a **cluster** because a single instance can serve and handle multiple databases. Every database is an isolated space where users and applications can store data.

A database is accessed by allowed users, but users connected to a database cannot cross the database boundaries and interact with data contained in another database unless they explicitly connect to the latter database too.

A database can be organized into namespaces, called *schemas*. A schema is a mnemonic name that the user can assign to organize database objects, such as tables, into a more structured collection. Schemas cannot be nested, so they represent a flat namespace.

Database objects are represented by everything the user can create and manage within the database—for instance, tables, functions, triggers, and data types. Every object belongs to one and only one schema that, if not specified, is named as the user that creates the object.



In PostgreSQL versions prior to 15, every new object belongs to the default public schema if not specified otherwise. Since PostgreSQL 15, every user is assigned a personal schema and objects belong to such a schema unless a different schema name is explicitly specified.

Users are defined at a cluster-wide level, which means they are not tied to a particular database in the cluster. A user can connect with and manage any database in the cluster they are allowed to.

PostgreSQL splits users into two main categories:

- **Normal users:** These users are the ones who can connect to and handle databases and objects depending on their privilege set.
- **Superusers:** These users can do anything with any database object.

PostgreSQL allows the configuration of as many superusers as you need, and every superuser has the very same permissions: they can do everything with every database and object and, most notably, can also control the life cycle of the cluster (for instance, they can terminate normal user connections, reload the configuration, stop the whole cluster, and so on).

PostgreSQL internal data, such as users, databases, namespaces, configuration, and database run-time status, is provided by means of **catalogs**: special tables and views that present information in a SQL-interactive way. Many catalogs are trimmed depending on the user who is inspecting them, with the exception that superusers usually see the whole set of available information.

PostgreSQL stores the user data (for example, tables) and its internal status on the local filesystem.